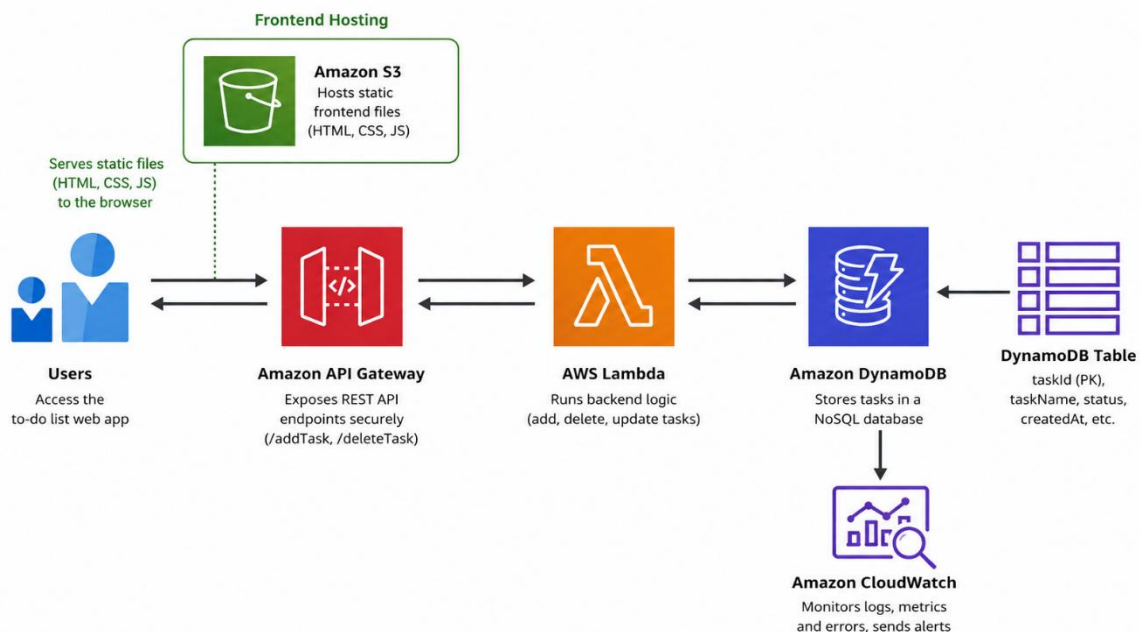


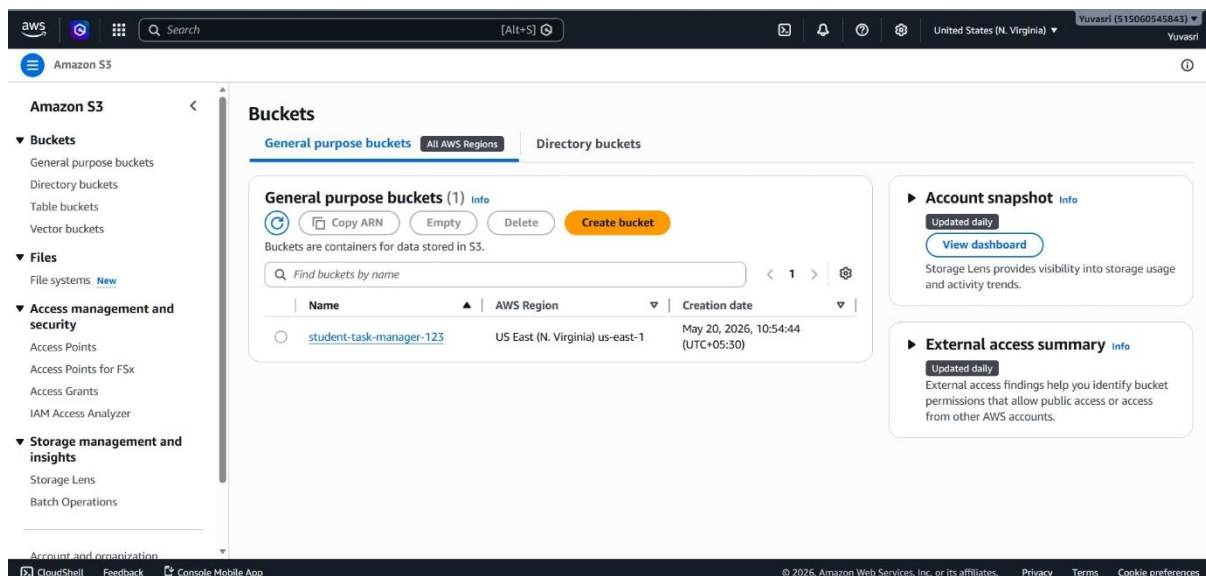
Overview of the Project and Screenshots for Outcomes

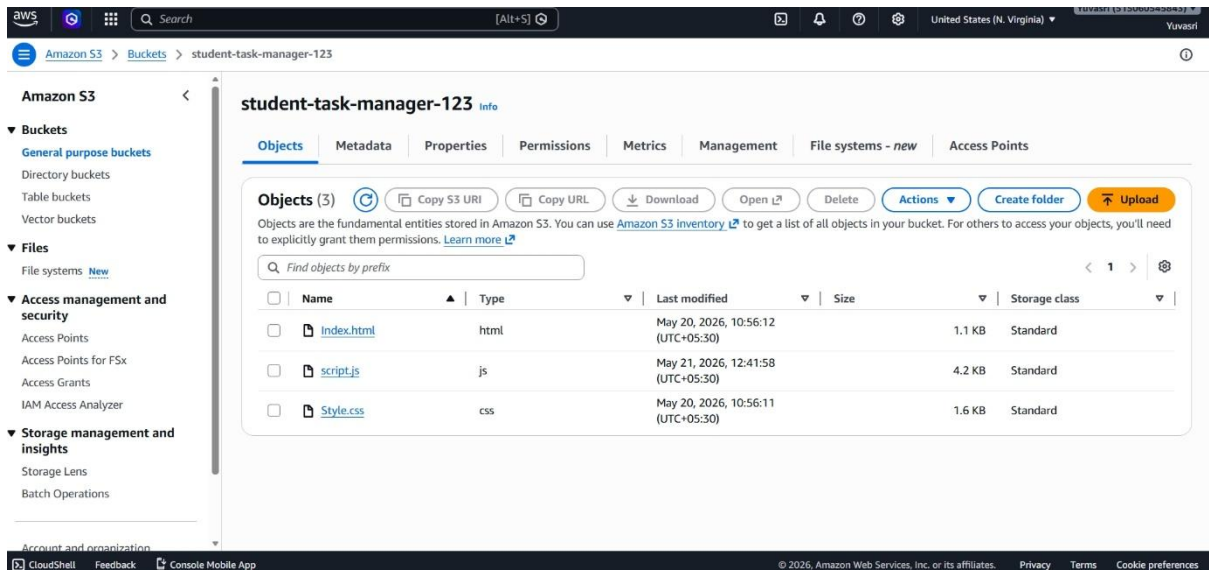


Flow chart : Scalable Serverless Web App Architecture on AWS

Hosting in Amazon S3 :

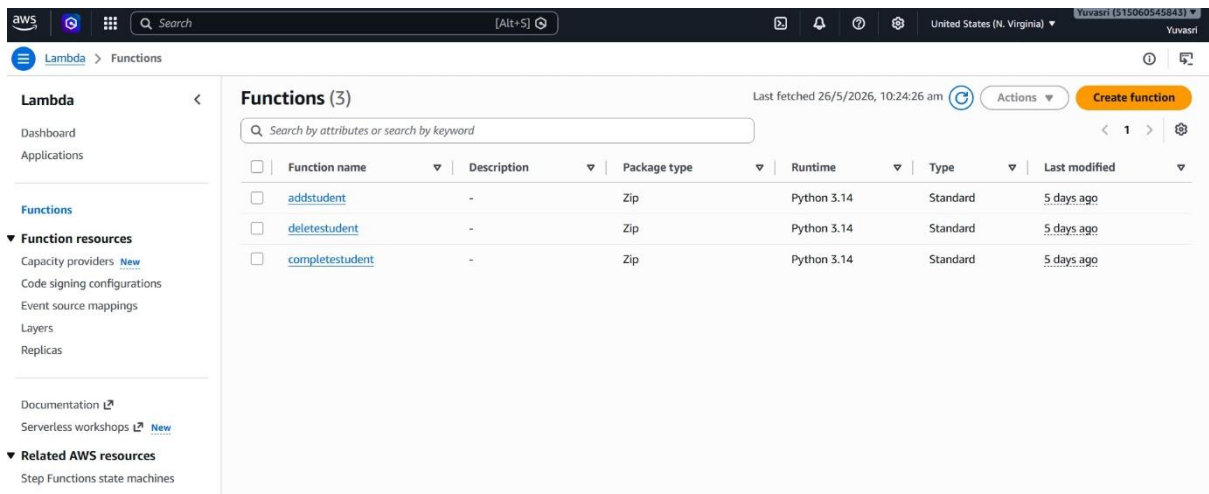
Stored the frontend files (HTML, CSS, and JavaScript) inside an Amazon S3 bucket. Enabled Static Website Hosting in S3 to serve the web application to users.





Lambda function:

AWS Lambda was used to run backend functions without managing servers and it handled the CRUD Operations.



Lambda add function with Integrated API

The screenshot displays the AWS Lambda console for the 'addstudent' function. The 'Function overview' section shows the function name, layers (0), and an API Gateway trigger. The 'Code source' section shows the Python code for the lambda handler, which interacts with a DynamoDB table named 'students' and returns a JSON response with a 'statusCode' of 200. The console also shows the function's ARN, description, and last modified date.

Function overview

Diagram | Template

addstudent

Layers (0)

API Gateway

+ Add trigger

+ Add destination

Function ARN: arn:aws:lambda:us-east-1:515060545843:function:addstudent

Description: -

Last modified: 6 days ago

Export to Infrastructure Composer | Download

Code | Test | Monitor | Configuration | Aliases | Versions

Code source

Open in Visual Studio Code | Update

```
6 table = dynamodb.Table('students')
7
8 def lambda_handler(event, context):
9
10     print("Event:", event)
11
12     body = json.loads(event.get('body', '{}'))
13
14     item = {
15         'taskId': str(uuid.uuid4()),
16         'name': body.get('name'),
17         'rollNo': body.get('rollNo'),
18         'course': body.get('course'),
19         'status': 'Pending'
20     }
```

PROBLEMS | OUTPUT | CODE REFERENCE LOG | TERMINAL

Status: Succeeded

Test Event Name: addstudent

Response:

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*"
  },
  "body": json.dumps({
    "message": "Student added successfully"
  })
}
```

Lambda delete function Integrated with API

The screenshot displays the AWS Lambda console for the 'deletestudent' function. The 'Function overview' section shows the function name, layers (0), and an API Gateway trigger. The 'Code source' section shows the Python code for the lambda handler, which interacts with a DynamoDB table named 'students' and returns a JSON response with a 'statusCode' of 200. The console also shows the function's ARN, description, and last modified date.

Function overview

Diagram | Template

deletestudent

Layers (0)

API Gateway

+ Add trigger

+ Add destination

Function ARN: arn:aws:lambda:us-east-1:515060545843:function:deletestudent

Description: -

Last modified: 6 days ago

Export to Infrastructure Composer | Download

Code | Test | Monitor | Configuration | Aliases | Versions

Code source

Open in Visual Studio Code | Update

```
8 def lambda_handler(event, context):
9
10     print(event)
11
12     return {
13         'statusCode': 200,
14         'headers': {
15             'Access-Control-Allow-Origin': '*',
16             'Access-Control-Allow-Headers': 'Content-Type',
17             'Access-Control-Allow-Methods': 'OPTIONS,POST'
18         },
19         'body': json.dumps({
20             'message': 'Student deleted successfully'
21         })
22     }
```

PROBLEMS | OUTPUT | CODE REFERENCE LOG | TERMINAL

Status: Succeeded

Test Event Name: delete

Response:

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*"
  },
  "body": json.dumps({
    "message": "Student deleted successfully"
  })
}
```

Lambda Completed function Integrated with API

The screenshot displays the AWS Lambda console for the 'completestudent' function. The 'Function overview' tab is active, showing a diagram of the function's architecture with an API Gateway trigger and no layers. The 'Code source' tab is also visible, showing the Python code for the function. The code defines a handler that returns a 200 status code and a JSON response with headers and a body. The 'Execution Results' section shows a successful test event with a response containing status code 200, headers, and a body.

```
def handler(event, context):  
    try:  
        # Your logic here  
        return {  
            'statusCode': 200,  
            'headers': {  
                'Access-Control-Allow-Origin': '*',  
            },  
            'body': json.dumps({  
                'error': str(e)  
            })  
        }  
    except Exception as e:  
        return {  
            'statusCode': 500,  
            'headers': {  
                'Access-Control-Allow-Origin': '*',  
            },  
            'body': json.dumps({  
                'error': str(e)  
            })  
        }
```

Dynamodb database for data storage

It is a database for this project. Creating a table in Dynamodb and exploring the dataset.

The screenshot displays the AWS DynamoDB console. The 'Tables' section is active, showing a list of tables. The 'students' table is listed with the following details:

Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection	Favorite	Read capacity m
students	Active	taskid (S)	-	0	0	Off		On-demand

aws [Search] [Alt+S] United States (N. Virginia) Yuvasri (S15060545843) Yuvasri

DynamoDB > Explore items > students

DynamoDB

- Dashboard
- Tables
- Explore items
- PartiQL editor
- Backups
- Exports to S3
- Imports from S3
- Integrations
- Reserved capacity
- Settings

▼ DAX

- Clusters
- Subnet groups
- Parameter groups
- Events

Filter by tag value
Any tag value

Find tables

students

Scan Query

Select a table or index
Table - students

Select attribute projection
All attributes

Filters - optional

Run Reset

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2

Table: students - Items returned (1)

Scan started on May 26, 2026, 10:21:46

taskid (String)	course	name	rollNo	status
0bfb5714-dfe0-4531...	AWS	yuvasri	01	Completed

Amazon CloudWatch was used to monitor task activity, logs, and application errors.

aws [Search] [Alt+S] United States (N. Virginia) Yuvasri (S15060545843) Yuvasri

CloudWatch > Log management

CloudWatch

- Favorites and recents
- AI Operations
- GenAI Observability
- Application Signals (APM)
- Infrastructure Monitoring
- ▼ Logs
 - Log Management
 - Log Analytics Preview
 - Log Anomalies
 - Live Tail
 - Logs Insights
 - Contributor Insights
- ▼ Metrics
 - All metrics
 - Query Studio Preview

Log groups Data sources - new Summary

Get-set-go with unified data store

Log groups (3)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

Log group	Log class	Anomaly d...	Deleti...	Bearer...	Dat...	Sen...	Retenti...
/aws/lambda/addstudent	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/completestudent	Standard	Configure	Off	Off	-	-	Never exp...
/aws/lambda/deletestudent	Standard	Configure	Off	Off	-	-	Never exp...